OAVS TGT COMPUTER SCIENCE TEACHER EXAMINATION

MCQ Question Bank

Built-in Operators and Functions

Console-Based Input/Output and Related Built-in I/O Functions

printf() scanf() getch() getchar() putchar() gets() puts()

Question Bank Overview

Total Unique MCQs: **141 Questions**

Question Types Covered:

Definition | Concept | Syntax | Function Identification | Output-Based | Fill-in-the-Blank | Statement-Based | Assertion-Reason | Match-the-Following | Comparison | Error Identification | Code Snippet | Scenario-Based | Previous-Year Pattern | Difficult/Tricky

Section-wise Question Count:

- printf(): 43
- scanf(): 31
- getchar(): 12
- getch(): 10
- putchar(): 11
- gets(): 11
- puts(): 10
- Comparison & Mixed Concepts: 13

Each question includes the correct answer and a concise, exam-oriented explanation.
Strictly limited to the prescribed syllabus: Console-Based I/O and the seven built-in functions listed above.

Table of Contents

Section	Questions	Page
1. printf()	43	3
2. scanf()	31	18
3. getchar()	12	29
4. getch()	10	33
5. putchar()	11	37
6. gets()	11	41
7. puts()	10	45
8. Comparison & Mixed Concepts	13	49

printf() — 43 Questions

Q1 Definition printf() - Purpose

What is the primary purpose of the printf() function in C?

A. To display formatted output on the console

B. To read formatted input from the keyboard

C. To read a single character from input

D. To allocate memory dynamically

Ans: A printf() is a standard library output function used to display formatted text and variable values on the console.

Q2 Definition printf() - Header File

Which header file must be included to use printf() in a C program?

A. conio.h

B. stdlib.h

C. stdio.h

D. string.h

Ans: C printf() is declared in stdio.h (Standard Input Output header), which must be included for the compiler to recognize it.

Q3 Syntax printf() - Syntax

Which of the following is the correct general syntax of printf()?

A. printf["format string"];

B. print("format string");

C. printf("format string", argument_list);

D. printf(variable);

Ans: C The correct syntax is printf("format string", argument_list); where the format string contains format specifiers corresponding to the arguments.

Q4 Concept printf() - Return Value

What does printf() return on successful execution?

A. The number of characters successfully printed

B. The first character printed

C. Nothing, it is a void function

D. The value 0

Ans: A printf() returns an integer equal to the number of characters successfully printed; it returns a negative value on failure.

Q5 Function Identification printf() - Format Specifier

Which format specifier is used in printf() to print an integer value?

A. %f

B. %s

C. %d

D. %c

Ans: C %d is the format specifier used to print signed decimal integers in printf().

Q6 Function Identification printf() - Format Specifier

Which format specifier is used in printf() to print a floating-point value?

A. %f

B. %c

C. %d

D. %u

Ans: A %f is used as the format specifier to print floating-point (real) numbers using printf().

Q7 Function Identification printf() - Format Specifier

Which format specifier is used in printf() to print a single character?

A. %f

B. %d

C. %c

D. %s

Ans: C %c is the format specifier used to print a single character value in printf().

Q8 Function Identification printf() - Format Specifier

Which format specifier is used in printf() to print a string?

A. %d

B. %s

C. %c

D. %ld

Ans: B %s is the format specifier used to print a string (a sequence of characters terminated by a null character) using printf().

Q9 Function Identification printf() - Format Specifier

Which format specifier is used to print an unsigned integer using printf()?

A. %d

B. %u

C. %f

D. %c

Ans: B %u is used to print unsigned integer values, which do not have a sign bit and represent only non-negative numbers.

Q10 Function Identification printf() - Format Specifier

Which format specifier prints a value in hexadecimal form using printf()?

A. %d

B. %e

C. %x

D. %o

Ans: C %x is used to print an integer value in hexadecimal (base 16) form using lowercase letters a-f.

Q11 Function Identification printf() - Format Specifier

Which format specifier prints a value in octal form using printf()?

A. %x

B. %o

C. %u

D. %d

Ans: B %o is used to print an integer value in octal (base 8) form using printf().

Q12 Function Identification printf() - Format Specifier

Which format specifier is used to print a value in scientific (exponential) notation?

A. %f

B. %e

C. %g

D. %c

Ans: B %e is used to display a floating-point number in exponential (scientific) notation, e.g., 1.500000e+01.

Q13 Function Identification printf() - Format Specifier

Which format specifier is used to print a long integer using printf()?

A. %lc

B. %ls

C. %ld

D. %lf

Ans: C %ld is the correct format specifier to print a value of type long int using printf().

Q14 Output-Based printf() - Escape Sequence

What is the output of: printf("Hello\nWorld");

A. Hello World on the same line

B. Hello followed by World on a new line

C. An error message

D. HelloWorld with no space

Ans: B The escape sequence \n moves the cursor to a new line, so 'Hello' and 'World' are printed on two separate lines.

Q15 Output-Based printf() - Escape Sequence

What is the output of: printf("A\tB");

A. AB with no space

B. A syntax error

C. A and B separated by a tab space

D. A new line between A and B

Ans: C The escape sequence \t inserts a horizontal tab space between the characters A and B.

Q16 Concept printf() - Escape Sequence

Which escape sequence is used in printf() to print a backslash character?

A. \\

B. \0

C. \b

D. \n

Ans: A The escape sequence \\ is used to print a single literal backslash character on the screen.

Q17 Concept printf() - Escape Sequence

Which escape sequence in printf() produces an audible alert/beep sound?

A. \b

B. \f

C. \a

D. \v

Ans: C The escape sequence \a represents the alert character, which produces a beep sound on most systems.

Q18 Concept printf() - Escape Sequence

Which escape sequence is used in printf() to print a double quote character?

A. \?

B. \n

C. \'

D. \"

Ans: D The escape sequence \" is used inside a printf() string to print a literal double quote character.

Q19 Output-Based printf() - Field Width

What does the format specifier %5d in printf() indicate?

A. Print an integer with a minimum field width of 5

B. Print 5 decimal places

C. Print an integer 5 times

D. Print 5 integers

Ans: A The number before 'd' specifies the minimum field width; the value is right-justified and padded with spaces if shorter than 5 characters.

Q20 Output-Based printf() - Precision

What does %.2f specify in a printf() statement?

A. Print the value twice

B. Print only 2 digits before the decimal point

C. Print a floating-point number with exactly 2 digits after the decimal point

D. Print 2 floating values

Ans: C The .2 precision specifier limits the number of digits printed after the decimal point to exactly two for a floating-point value.

Q21 Output-Based printf() - Left Justification

What is the effect of using a minus sign in a format specifier, e.g. %-10d, in printf()?

A. Left-justifies the output within the given field width

B. Causes a compilation error

C. Right-justifies the output

D. Prints the value as negative

Ans: A The minus sign before the field width left-justifies the printed value, padding spaces on the right instead of the left.

Q22 Comparison printf() vs puts()

Which of the following is a key advantage of printf() over puts()?

A. printf() is faster than puts() for all cases

B. printf() automatically adds a newline, puts() does not

C. printf() can print multiple data types using format specifiers, puts() can only print strings

D. puts() cannot be used in C programs

Ans: C printf() supports multiple format specifiers (%d, %f, %c, %s, etc.) and can print various data types, whereas puts() can only output a single string.

Q23 Comparison printf() vs puts()

Which statement correctly differentiates printf() from puts()?

A. Both functions behave identically in all cases

B. puts() can print integers directly without conversion

C. printf() requires a format string; puts() simply prints a string followed by a newline

D. puts() requires a format string; printf() does not

Ans: C printf() needs a format string with specifiers to interpret arguments, while puts() takes only a string argument and outputs it followed by a newline automatically.

Q24 Concept printf() - Multiple Arguments

In the statement printf("%d %d", a, b);, what is required for correct output?

A. printf() automatically ignores extra arguments without errors

B. The number of format specifiers must match the number of arguments supplied

C. Only one argument can be passed to printf() at a time

D. The arguments must be passed in reverse order

Ans: B Each format specifier in the format string must correspond to one argument; mismatched counts/types can lead to undefined or garbage output.

Q25 Error Identification printf() - Common Mistake

What is the likely outcome of the statement: printf("%d", "Hello");

A. It compiles and runs perfectly with no issues

B. It prints Hello correctly

C. It automatically converts the string to an integer

D. It leads to undefined or garbage output because of a format specifier and argument type mismatch

Ans: D Using %d with a string argument causes a type mismatch, resulting in undefined behavior or garbage values being printed, since printf() does not perform automatic type checking at runtime.

Q26 Error Identification printf() - Common Mistake

What is wrong with the statement: printf("Value = %d" x);

A. A comma is missing between the format string and the variable x

B. printf() cannot accept variables

C. Nothing, it is correct

D. %d should be %f

Ans: A Arguments in printf() must be separated by commas; the missing comma between the format string and x causes a syntax error.

Q27 Assertion-Reason printf() - Header File

Assertion (A): printf() can be used without including stdio.h in standard C.
Reason (R): printf() is a built-in keyword of the C language.
Choose the correct option:

A. A is true, but R is false

B. A is false, but R is true

C. Both A and R are true, and R correctly explains A

D. Both A and R are false

Ans: D printf() is a library function, not a keyword, and standard C requires including stdio.h for its proper declaration and usage.

Q28 Assertion-Reason printf() - Return Value

Assertion (A): printf() returns the number of characters printed.
Reason (R): printf() is declared as a function returning type int.
Choose the correct option:

A. A is true, but R is false

B. Both A and R are true, and R correctly explains A

C. A is false, but R is true

D. Both A and R are false

Ans: B printf() is declared with return type int, and it returns the count of characters successfully written, which justifies why R correctly explains A.

Q29 Fill in the Blank printf() - Format Specifier

Fill in the blank: To print a character variable using printf(), the format specifier _____ should be used.

A. %f

B. %s

C. %c

D. %d

Ans: C %c is specifically used as the format specifier for printing a single character value.

Q30 Fill in the Blank printf() - Header File

Fill in the blank: printf() is declared in the _____ header file.

A. conio.h

B. stdio.h

C. string.h

D. math.h

Ans: B The function prototype of printf() is present in the stdio.h header file, which is why it must be included before use.

Q31 Statement-Based printf() - Behaviour

Which of the following statements about printf() is TRUE?

A. printf() cannot print negative numbers

B. printf() automatically inserts a newline after printing

C. printf() does not add any newline automatically; \n must be used explicitly

D. printf() can only print one value at a time

Ans: C Unlike puts(), printf() does not automatically move to a new line; the programmer must explicitly include \n in the format string.

Q32 Statement-Based printf() - Behaviour

Which statement about the format string in printf() is correct?

A. The format string is optional in every printf() call

B. The format string must always be a variable, not a literal

C. The format string can only contain format specifiers, no plain text

D. The format string may contain plain text, escape sequences, and format specifiers together

Ans: D The format string in printf() can combine literal text, escape sequences (like \n), and format specifiers (like %d) in any order.

Q33 Code Snippet printf() - Output Tracing

What will be the output of the following code?

```
int a = 5;
printf("%d", a*2);
```

A. 10

B. 5

C. Error

D. 52

Ans: A The expression a*2 evaluates to 10 before being passed to printf(), so the value 10 is printed using the %d specifier.

Q34 Code Snippet printf() - Output Tracing

What will be the output of the following code?
printf("%d %d", 10, 20);

A. 10 20

B. 1020

C. Error

D. 20 10

Ans: A The two %d specifiers correspond to 10 and 20 respectively, printed in order with a space as per the format string, giving '10 20'.

Q35 Code Snippet printf() - Output Tracing

What will be the output of the following code?
char ch = 'A';
printf("%c", ch);

A. Error

B. ch

C. 65

D. A

Ans: D The %c specifier prints the character corresponding to the variable ch, which holds the character 'A', so 'A' is displayed.

Q36 Code Snippet printf() - Output Tracing

What will be the output of the following code?
char ch = 'A';
printf("%d", ch);

A. 0

B. Error

C. A

D. 65

Ans: D When a character is printed using %d, its underlying ASCII value is displayed; the ASCII value of 'A' is 65.

Q37 Scenario-Based printf() - Practical Use

A programmer wants to display the message 'Result: 90' where 90 is stored in an integer variable called marks. Which statement is correct?

A. printf("Result: ", marks);

B. printf("Result: %s", marks);

C. printf(Result: %d, marks);

D. printf("Result: %d", marks);

Ans: D The format string must include %d to correctly substitute and display the integer value stored in marks after the text 'Result: '.

Q38 Previous-Year Pattern printf() - General

In OAVS/competitive exams, printf() is most commonly classified as which type of function?

A. String manipulation function only

B. Mathematical function

C. User-defined function

D. Standard library output function

Ans: D printf() is part of the C standard library and is categorized as an output function used for displaying data on the console.

Q39 Difficult/Tricky printf() - Tricky Output

What will printf("%d\n", 5/2); print?

A. 3

B. 2

C. 2.5

D. Error

Ans: B Since both operands are integers, 5/2 performs integer division, discarding the remainder, resulting in 2, which is then printed using %d.

Q40 Difficult/Tricky printf() - Tricky Output

What will be printed by: printf("%d", 'A' + 1);

A. 66

B. Error

C. B

D. A1

Ans: A 'A' has ASCII value 65; adding 1 gives 66, and since %d is used, the integer result 66 is printed instead of a character.

Q41 Concept printf() - Advantages

What is one major advantage of using printf() for output in C?

A. It requires no header file

B. It works only with character data

C. It allows formatted, mixed-type output using a single statement

D. It can only display integers

Ans: C printf() allows combining multiple data types and custom formatting (width, precision, alignment) in a single output statement, which is a major advantage.

Q42 Match the Following printf() - Format Specifiers

Match the format specifier with its purpose:
1. %d 2. %f 3. %c 4. %s
A. String B. Character C. Integer D. Float

A. 1-A, 2-B, 3-C, 4-D

B. 1-D, 2-C, 3-A, 4-B

C. 1-C, 2-D, 3-B, 4-A

D. 1-B, 2-A, 3-D, 4-C

Ans: C %d corresponds to Integer, %f to Float, %c to Character, and %s to String — the standard mapping of printf() format specifiers.

Q43 **Concept** **printf() - Working**

How does printf() internally process its arguments?

A. It ignores the format string and prints arguments in random order

B. It scans the format string and substitutes each format specifier with the corresponding argument value in sequence

C. It prints all arguments first and then the format string

D. It converts every argument to a string before printing regardless of specifier

Ans: B

printf() scans the format string from left to right, and wherever it finds a format specifier, it substitutes the next argument from the argument list in the order they were passed.

scanf() — 31 Questions

Q44 Definition scanf() - Purpose

What is the primary purpose of the scanf() function in C?

A. To display output on the console

B. To read a single character only

C. To clear the input buffer

D. To read formatted input from the standard input device

Ans: D scanf() is a standard library input function used to read formatted data of various types from the standard input device (keyboard).

Q45 Definition scanf() - Header File

Which header file is required to use the scanf() function?

A. conio.h

B. stdio.h

C. stdlib.h

D. string.h

Ans: B scanf() is declared in the stdio.h header file, which must be included before using it in a C program.

Q46 Syntax scanf() - Syntax

Which of the following represents the correct syntax of scanf()?

A. scanf("format string", &variable1, &variable2, ...);

B. scanf("format string");

C. scan("format string", variable);

D. scanf(variable, "format");

Ans: A The correct syntax is scanf("format string", &variable1, &variable2, ...); where the address operator & is used before each variable.

Q47 Concept scanf() - Address Operator

Why is the address operator (&) generally used before variable names in scanf()?

A. It is optional and has no real purpose

B. To pass the memory address of the variable so scanf() can store the input value directly into it

C. To convert the variable into a string

D. To declare the variable

Ans: B scanf() needs the memory address of a variable to store the input value directly at that location; the & operator provides this address.

Q48 Concept scanf() - Return Value

What does scanf() return?

A. The number of input items successfully read and assigned

B. Always returns 0

C. The value entered by the user

D. Always returns 1

Ans: A scanf() returns an integer indicating the number of input items that were successfully matched and assigned; it returns EOF on input failure before any conversion.

Q49 Function Identification scanf() - Format Specifier

Which format specifier is used in scanf() to read an integer value?

A. %c

B. %d

C. %f

D. %s

Ans: B %d is used in scanf() to read a signed decimal integer value from the input stream.

Q50 Function Identification scanf() - Format Specifier

Which format specifier is used in scanf() to read a floating-point value?

A. %s

B. %f

C. %c

D. %d

Ans: B %f is the format specifier used in scanf() to read a floating-point (real) number from input.

Q51 Function Identification scanf() - Format Specifier

Which format specifier is used in scanf() to read a single character?

A. %d

B. %c

C. %s

D. %f

Ans: B %c is used in scanf() to read exactly one character from the input, including whitespace if not handled carefully.

Q52 Function Identification scanf() - Format Specifier

Which format specifier is used in scanf() to read a string (without spaces)?

A. %s

B. %f

C. %d

D. %c

Ans: A %s reads a sequence of non-whitespace characters from input and automatically appends a null terminator, stopping at the first space.

Q53 Concept scanf() - Multiple Inputs

Which statement correctly reads two integers into variables a and b using scanf()?

A. scanf("%d%d", &a, &b);

B. scanf("%d %d", a, b);

C. scanf("%d %d", &a, &b);

D. scanf("%d, %d", &a, &b);

Ans: C To read multiple values, scanf() needs corresponding format specifiers and the address of each variable; "%d %d", &a, &b correctly reads two integers.

Q54 Error Identification scanf() - Common Mistake

What is the likely problem with the statement: scanf("%d", a);

A. scanf() cannot accept a single argument

B. The %d specifier is wrong for integers

C. The address operator (&) is missing before variable a, which can cause undefined behavior

D. No problem; it works correctly

Ans: C Without the & operator, scanf() does not receive the address of variable a, leading to undefined behavior since it cannot store the value correctly.

Q55 Error Identification scanf() - Common Mistake

Which of the following is a common mistake when using scanf() to read a string?

A. Forgetting to declare the character array

B. Using & before the array name unnecessarily, which works but is conceptually inconsistent

C. Using %s, which stops reading at the first whitespace, when a full sentence with spaces is expected

D. Both B and C are common mistakes

Ans: D %s only reads until the first whitespace, so multi-word input gets truncated; additionally, although arrays decay to pointers (so & is technically unnecessary and sometimes generates a warning), this confusion is a frequent beginner mistake.

Q56 Concept scanf() - Data Types

Which format specifier should be used in scanf() to read a double value?

A. %lf

B. %ld

C. %d

D. %f

Ans: A %lf is the correct format specifier for reading a double-precision floating-point value using scanf().

Q57 Concept scanf() - Data Types

Which format specifier is used in scanf() to read a long integer?

A. %lf

B. %ld

C. %lu

D. %lc

Ans: B %ld is used to read a value into a variable of type long int using scanf().

Q58 Comparison scanf() vs getchar()

Which of the following correctly differentiates scanf() from getchar()?

A. scanf() cannot read characters at all

B. getchar() requires a format string; scanf() does not

C. Both functions are identical in functionality

D. scanf() can read multiple formatted values; getchar() reads only a single character at a time

Ans: D scanf() is a general-purpose formatted input function that can read multiple values of different types, whereas getchar() is limited to reading a single character.

Q59 Comparison gets() vs scanf()

What is a key difference between gets() and scanf() when reading strings?

A. Both functions behave identically for string input

B. gets() requires a format specifier like scanf()

C. scanf() can read spaces but gets() cannot

D. gets() can read a string including spaces until a newline; scanf() with %s stops at the first whitespace

Ans: D gets() reads an entire line including embedded spaces until a newline is encountered, while scanf() with %s reads only a single 'word' and stops at the first whitespace.

Q60 Assertion-Reason scanf() - Address Operator

Assertion (A): The & operator is required before normal variables passed to scanf().
Reason (R): scanf() needs the memory address to store the input value at the correct location.
Choose the correct option:

A. Both A and R are true, and R correctly explains A

B. Both A and R are false

C. A is false, but R is true

D. A is true, but R is false

Ans: A scanf() stores input directly into memory, so it must be given the address of the variable; this is exactly why the & operator is required for ordinary variables.

Q61 Assertion-Reason scanf() - Return Value

Assertion (A): scanf() returns the number of successfully read and assigned input items.
Reason (R): scanf() always returns 1 regardless of input.
Choose the correct option:

A. Both A and R are false

B. A is false, but R is true

C. A is true, but R is false

D. Both A and R are true, and R correctly explains A

Ans: C A is correct because scanf() returns the count of successfully matched items, but R is false since the return value varies and is not always 1.

Q62 Fill in the Blank scanf() - Address Operator

Fill in the blank: To read an integer value into variable num using scanf(), the correct call is scanf("%d", _____ num);

A. &

B. $

C. #

D. *

Ans: A The & (address-of) operator must precede the variable name so that scanf() receives the memory address where the input value should be stored.

Q63 Fill in the Blank scanf() - Header File

Fill in the blank: scanf() is declared in the _____ header file.

A. math.h

B. conio.h

C. ctype.h

D. stdio.h

Ans: D Like printf(), scanf() is also declared in the stdio.h header, which must be included for input/output operations.

Q64 Statement-Based scanf() - Behaviour

Which statement about scanf() with the %d specifier is TRUE?

A. It treats spaces as part of the number

B. It automatically skips leading whitespace before reading the integer

C. It requires the & operator even for the format string

D. It cannot read negative numbers

Ans: B Numeric format specifiers like %d in scanf() automatically skip any leading whitespace (spaces, tabs, newlines) before reading the actual value.

Q65 Statement-Based scanf() - Behaviour

Which of the following statements correctly describes a limitation of %c in scanf()?

A. %c can only be used once per program

B. %c does NOT automatically skip leading whitespace, so it may read a leftover newline character from a previous input

C. %c automatically skips whitespace before reading a character, unlike %d

D. %c cannot be used with scanf() at all

Ans: B Unlike numeric specifiers, %c reads the very next character including whitespace, which often causes it to capture a leftover newline from a previous scanf() call.

Q66 Code Snippet scanf() - Output Tracing

If the user enters '25' for the following code, what is stored in num?

```
int num;
scanf("%d", &num);
```

A. Nothing is stored

B. 25 as an integer value

C. '25' as a string

D. An error occurs

Ans: B scanf() with %d reads the input characters '25', converts them to the integer value 25, and stores it at the memory address of num.

Q67 Code Snippet scanf() - Output Tracing

What happens if the user enters 'abc' for the following code?

```
int num;
scanf("%d", &num);
```

A. 'abc' is converted to its ASCII sum and stored in num

B. num is set to 0 automatically and the program continues normally

C. The matching fails because 'abc' is not a valid integer, leaving num uninitialized and scanf() returns a value less than 1

D. The program crashes immediately

Ans: C Since %d expects digits, non-numeric input like 'abc' cannot be converted; scanf() fails to make an assignment, num remains unmodified, and the return value reflects the failed match.

Q68 Scenario-Based scanf() - Practical Use

A program needs to take the user's age (an integer) as input and store it in a variable called age. Which statement is correct?

A. scanf("%s", &age);

B. scanf("%d", &age);

C. scanf("%d", age);

D. scanf(age, "%d");

Ans: B To correctly store an integer input, scanf() must use the %d specifier along with the address of the variable, written as &age.

Q69 Previous-Year Pattern scanf() - General

In exam-style questions, scanf() is best described as which type of function?

A. An output-only function

B. A mathematical computation function

C. A formatted standard input function

D. A character-only input function

Ans: C scanf() is categorized as a standard, formatted input function capable of reading multiple data types based on format specifiers.

Q70 Difficult/Tricky scanf() - Tricky Concept

Why can mixing scanf("%d", &n); followed by scanf("%c", &ch); cause unexpected results?

A. Because the newline character left in the input buffer after entering the integer is read by %c instead of waiting for new input

B. Because %c always skips one extra character automatically

C. Because scanf() does not support character input after integer input

D. Because %d cannot be used before %c under any circumstance

Ans: A After reading an integer, the newline character (from pressing Enter) remains in the input buffer; since %c does not skip whitespace, it reads this leftover newline instead of waiting for a fresh character.

Q71 Comparison — Formatted vs Unformatted I/O

Why is scanf() considered a 'formatted' input function while getchar() is considered 'unformatted'?

A. getchar() uses format specifiers and scanf() does not

B. Neither function is related to formatting

C. scanf() uses a format string with specifiers to interpret different data types, while getchar() simply reads one character with no format string

D. Both are formatted input functions

Ans: C scanf() relies on a format string with specifiers (%d, %f, %c, %s) to interpret various data types, whereas getchar() performs simple unformatted single-character input with no format string.

Q72 Concept — scanf() - Advantages

What is a key advantage of scanf() over single-purpose input functions like getchar()?

A. scanf() cannot read floating-point numbers

B. scanf() is restricted to only one variable per call

C. scanf() can read multiple different data types in a single call using a format string

D. scanf() works only with character arrays

Ans: C scanf() allows reading several values of mixed data types in one statement by listing multiple format specifiers and corresponding addresses.

Q73 Match the Following — scanf() - Format Specifiers

Match the specifier with the data type it reads in scanf():
1. %d 2. %f 3. %c 4. %s
A. String B. Character C. Float D. Integer

A. 1-D, 2-C, 3-B, 4-A

B. 1-B, 2-D, 3-A, 4-C

C. 1-A, 2-B, 3-C, 4-D

D. 1-C, 2-A, 3-D, 4-B

Ans: A %d reads Integer, %f reads Float, %c reads Character, and %s reads String — matching the standard scanf() specifier-to-type mapping.

Q74 **Concept** **scanf() - Working**

How does scanf() process the input stream when reading with %d?

A. It reads the entire line as a string and discards it

B. It ignores the input buffer entirely

C. It reads exactly one character regardless of type

D. It skips leading whitespace, then reads characters as long as they form a valid integer, stopping at the first non-digit character

Ans: D

scanf() with %d first skips any leading whitespace, then reads consecutive digit characters (optionally with a sign) to form an integer, stopping when a non-numeric character is encountered.

getchar() — 12 Questions

Q75 Definition getchar() - Purpose

What is the primary purpose of the getchar() function?

A. To read an entire string from input

B. To read a single character from the keyboard, including the Enter key in the buffer

C. To clear the screen

D. To display a character on the screen

Ans: B getchar() reads and returns a single character from the standard input stream, typically used to take simple character-based input.

Q76 Syntax getchar() - Syntax

Which of the following is the correct syntax to use getchar()?

A. getchar(ch);

B. char ch = getchar();

C. getchar("%c", ch);

D. char ch = getchar("input");

Ans: B getchar() takes no arguments and returns the character read, so the correct usage is to assign its return value to a character variable, as in char ch = getchar();.

Q77 Concept getchar() - Return Value

What does getchar() return when the end of input (EOF) is reached?

A. The null character '\0'

B. 0

C. It causes a program crash

D. -1 (typically defined as EOF)

Ans: D getchar() returns the special constant EOF (commonly -1) when the end of the input stream is reached, signaling no more characters are available.

Q78 Concept **getchar() - Header File**

Which header file declares the getchar() function?

A. conio.h

B. stdio.h

C. stdlib.h

D. ctype.h

Ans: B getchar() is part of the standard C library and is declared in the stdio.h header file.

Q79 Concept **getchar() - Echoing**

Does getchar() display (echo) the typed character on the screen?

A. No, it never echoes the character

B. Yes, it echoes the typed character on the screen as it is entered

C. Only on certain operating systems

D. It echoes only special characters

Ans: B getchar() is a standard input function that echoes the character typed by the user back to the screen as part of normal terminal behavior.

Q80 Concept **getchar() - Enter Key**

Does getchar() require the user to press the Enter key before the character is processed by the program?

A. getchar() disables the Enter key

B. getchar() works only with function keys

C. No, it processes the character instantly without Enter

D. Yes, because input is typically line-buffered, so Enter is needed to send the character to the program

Ans: D Because standard input is usually line-buffered, the character typed is held until the Enter key is pressed, after which getchar() retrieves it from the buffer.

Q81 Applications getchar() - Use Case

In which scenario is getchar() most useful?

A. Displaying large blocks of text

B. Reading a single confirmation character such as 'y' or 'n' from the user

C. Reading formatted multiple values with mixed data types

D. Performing complex mathematical calculations

Ans: B getchar() is commonly used for simple tasks such as reading a single confirmation character (e.g., yes/no responses) from the user.

Q82 Comparison getchar() vs getch()

What is the main difference between getchar() and getch()?

A. getchar() requires conio.h while getch() requires stdio.h

B. getchar() echoes the typed character and waits for Enter, while getch() reads a character immediately without echoing or waiting for Enter

C. getch() is part of standard C while getchar() is not

D. Both functions behave exactly the same

Ans: B getchar() is a standard function that echoes input and is line-buffered (waits for Enter), whereas getch() (non-standard, found in conio.h) reads a character immediately without echoing or requiring Enter.

Q83 Fill in the Blank getchar() - Header File

Fill in the blank: getchar() is declared in the _____ header file.

A. stdio.h

B. math.h

C. conio.h

D. stdlib.h

Ans: A getchar() belongs to the standard C library and is declared in stdio.h, just like printf() and scanf().

Q84 Code Snippet getchar() - Output Tracing

What will the following code do?

```
char c;
c = getchar();
putchar(c);
```

A. It deletes the last typed character

B. It prints a character without reading any input

C. It reads a character from input and immediately prints the same character using putchar()

D. It throws a compilation error

Ans: C getchar() reads one character entered by the user and stores it in c; putchar(c) then outputs that same character back to the screen.

Q85 Assertion-Reason getchar() - Behaviour

Assertion (A): getchar() can only be used to read alphabetic characters.
Reason (R): getchar() reads any single character including digits, symbols, and whitespace.
Choose the correct option:

A. A is true, but R is false

B. Both A and R are false

C. Both A and R are true, and R correctly explains A

D. A is false, but R is true

Ans: D A is false because getchar() is not restricted to alphabets — it reads any character at all, including digits, punctuation, and whitespace, which is exactly what R correctly states.

Q86 Statement-Based getchar() - Behaviour

Which of the following statements about getchar() is TRUE?

A. getchar() is defined only in conio.h

B. getchar() requires a format specifier

C. getchar() can read an entire word at once

D. getchar() takes no parameters and returns an int representing the character read

Ans: D getchar() is a parameterless function that returns an int (so it can also represent EOF), even though the value typically corresponds to a single character.

getch() — 10 Questions

Q87 Definition getch() - Purpose

What is the primary purpose of the getch() function?

A. To clear the input buffer

B. To print a character on the screen

C. To read an entire line of text

D. To read a character from the keyboard without echoing it and without waiting for Enter

Ans: D getch() reads a single character directly from the keyboard without displaying (echoing) it on the screen and without requiring the Enter key to be pressed.

Q88 Concept getch() - Header File

Which header file is typically required to use getch() (in Turbo C / older compilers)?

A. stdio.h

B. stdlib.h

C. ctype.h

D. conio.h

Ans: D getch() is a non-standard function commonly provided through conio.h in compilers like Turbo C, rather than being part of the ANSI C standard library.

Q89 Concept getch() - Echo Behaviour

Does getch() display the character typed by the user on the screen?

A. Yes, it always echoes the character like getchar()

B. It echoes the character only after pressing Enter

C. It echoes only numeric characters

D. No, getch() reads the character silently without displaying it

Ans: D A defining feature of getch() is that it captures the keystroke without echoing it to the screen, which is why it is often used for password input.

Q90 Concept — getch() - Enter Key Requirement

Does getch() require the user to press Enter before the character is read by the program?

A. getch() requires Enter only for special characters

B. No, getch() returns the character as soon as a key is pressed, without waiting for Enter

C. getch() ignores the Enter key always

D. Yes, exactly like getchar()

Ans: B getch() is unbuffered and reads a keypress immediately, unlike getchar(), which usually waits for the Enter key due to line buffering.

Q91 Applications — getch() - Use Case

In which scenario is getch() commonly used?

A. Performing arithmetic operations

B. Printing large reports

C. Declaring global variables

D. Pausing a program until any key is pressed, or accepting password input without showing characters

Ans: D getch() is frequently used to pause program execution (e.g., 'Press any key to continue...') or to read sensitive input like passwords without displaying it on screen.

Q92 Comparison — getchar() vs getch()

Which of the following best compares getchar() and getch() in terms of standardization?

A. Both are equally standard across all C compilers

B. Neither function is recognized in C programming

C. getchar() is a standard ANSI C function, whereas getch() is a non-standard function typically found in compiler-specific libraries like conio.h

D. getch() is part of ANSI C and getchar() is not

Ans: C getchar() is defined in the ANSI C standard (stdio.h), while getch() is non-standard and depends on compiler-specific headers such as conio.h, commonly used in DOS-based compilers.

Q93 Fill in the Blank getch() - Echo Behaviour

Fill in the blank: Unlike getchar(), the getch() function reads a character _____ echoing it on the screen.

A. only after

B. after

C. without

D. while

Ans: C getch() reads the character without echoing it, meaning the typed character is not displayed back on the screen automatically.

Q94 Statement-Based getch() - Behaviour

Which statement correctly describes a key behavioral feature of getch()?

A. It can only be used to read digits

B. It requires a format string like printf()

C. It automatically converts input to uppercase

D. It returns immediately upon a single keypress, without needing the Enter key

Ans: D A key characteristic of getch() is that it captures and returns a key as soon as it is pressed, without requiring Enter to be hit afterward.

Q95 Assertion-Reason getch() - Standardization

Assertion (A): getch() is part of the ISO/ANSI C standard library.
Reason (R): getch() is commonly provided via conio.h, which is specific to certain compilers like Turbo C.
Choose the correct option:

A. A is false, but R is true, and R explains why A is false

B. A is true, but R is false

C. Both A and R are true, and R correctly explains A

D. Both A and R are false

Ans: A getch() is NOT part of standard ANSI C; it is commonly supplied via the non-standard conio.h header in specific compilers, which is precisely why A is false while R remains true.

Q96 **Code Snippet** **getch() - Output Tracing**

What does the following code most likely intend to do?

```
printf("Press any key to continue...");
getch();
```

A. Pause the program execution until the user presses any key, without echoing or needing Enter

B. Terminate the program immediately

C. Read a string of unlimited length

D. Display the pressed key on the screen

Ans: A

This is a classic use of getch() to pause program execution until any key is pressed, commonly seen at the end of console programs.

putchar() — 11 Questions

Q97 Definition putchar() - Purpose

What is the primary purpose of the putchar() function?

A. To read a string from input

B. To read a single character from input

C. To write/display a single character to the standard output

D. To compare two characters

Ans: C putchar() is a standard output function used to write a single character to the console (standard output).

Q98 Syntax putchar() - Syntax

Which of the following is the correct syntax for using putchar()?

A. putchar[ch];

B. ch = putchar();

C. putchar("%c", ch);

D. putchar(ch);

Ans: D putchar() takes a single character (or its int representation) as an argument, written simply as putchar(ch);, without needing a format string.

Q99 Concept putchar() - Header File

Which header file declares putchar()?

A. stdlib.h

B. stdio.h

C. string.h

D. conio.h

Ans: B putchar() is a standard library function declared in stdio.h, the same header used for printf() and scanf().

Q100 | **Concept** | **putchar() - Return Value**

What does putchar() return after successfully writing a character?

A. Always returns void

B. Always returns 0

C. The ASCII value of the next character

D. The character written, as an int

Ans: D putchar() returns the character written (as an int) on success, or EOF if an output error occurs.

Q101 | **Applications** | **putchar() - Use Case**

In which scenario is putchar() typically used?

A. Performing complex string formatting

B. Reading multiple values at once

C. Allocating dynamic memory

D. Displaying one character at a time, such as in loops printing characters individually

Ans: D putchar() is commonly used when characters need to be output one at a time, such as inside a loop that processes and prints characters individually.

Q102 | **Comparison** | **puts() vs putchar()**

What is the key difference between puts() and putchar()?

A. putchar() outputs an entire string, while puts() outputs a single character

B. puts() outputs an entire string followed by a newline, while putchar() outputs only a single character

C. puts() requires a loop, but putchar() does not

D. Both functions are used only for numeric output

Ans: B puts() is designed to output a complete string (and automatically appends a newline), whereas putchar() is limited to outputting just one character at a time.

Q103 Fill in the Blank putchar() - Header File

Fill in the blank: putchar() is declared in the _____ header file.

A. ctype.h

B. stdlib.h

C. conio.h

D. stdio.h

Ans: D Like printf(), scanf(), and getchar(), putchar() is also part of the stdio.h standard header file.

Q104 Code Snippet putchar() - Output Tracing

What will the following code print?

```
putchar('X');
```

A. 'X' with quotes

B. X

C. It prints nothing because putchar() needs a format string

D. 120 (the ASCII value of X)

Ans: B putchar() directly outputs the character passed to it, so putchar('X'); simply prints the character X to the screen.

Q105 Code Snippet putchar() - Output Tracing

What will the following loop print?

```
for (char c = 'A'; c <= 'C'; c++) {
putchar(c);
}
```

A. An error message

B. ABC with no spaces

C. A B C with spaces

D. CBA

Ans: B The loop calls putchar() for 'A', 'B', and 'C' in sequence without inserting any spaces or newlines, producing the output ABC.

Q106 **Statement-Based** **putchar() - Behaviour**

Which of the following statements about putchar() is TRUE?

A. putchar() outputs exactly one character and does not add any newline automatically

B. putchar() requires the conio.h header file

C. putchar() automatically inserts a newline after the character

D. putchar() can only output digits, not letters

Ans: A putchar() outputs precisely one character to the screen and does not add a newline on its own; any newline must be added explicitly using '\n' or another putchar('\n') call.

Q107 **Assertion-Reason** **putchar() - Return Type**

Assertion (A): putchar() returns an int, not a char.
Reason (R): Returning an int allows putchar() to also represent the special EOF value on failure.
Choose the correct option:

A. A is false, but R is true

B. A is true, but R is false

C. Both A and R are true, and R correctly explains A

D. Both A and R are false

Ans: C putchar() is declared to return int specifically so that, in addition to normal character values, it can also return the special constant EOF to indicate an output error.

gets() — 11 Questions

Q108 Definition gets() - Purpose

What is the primary purpose of the gets() function in C?

A. To read a complete line of text (including spaces) from the standard input until a newline

B. To compare two strings

C. To display a string on the screen

D. To read a single character from input

Ans: A gets() reads an entire line of text from standard input, including embedded spaces, and stops only when a newline character is encountered.

Q109 Syntax gets() - Syntax

Which of the following is the correct syntax for using gets()?

A. str = gets;

B. gets("%s", str);

C. gets[str];

D. gets(str);

Ans: D gets() takes a single argument — the character array (string) where input should be stored — written as gets(str);, with no format specifier needed.

Q110 Concept gets() - Header File

Which header file declares the gets() function?

A. stdio.h

B. conio.h

C. string.h

D. stdlib.h

Ans: A gets() is declared in stdio.h, the same header file used for other standard input/output functions like printf() and scanf().

Q111 Concept gets() - Working

How does gets() determine when to stop reading characters from input?

A. It stops after reading a fixed number of characters

B. It stops when a newline character ('\n') is encountered, replacing it with a null terminator

C. It never stops automatically

D. It stops after reading exactly one word

Ans: B gets() continues reading characters from input until a newline is found; it then replaces the newline with a null character ('\0') to terminate the string.

Q112 Concept gets() - Limitation

What is a major limitation/danger of using gets()?

A. It can only read numeric input

B. It cannot read any input at all

C. It requires a format specifier, making it complex to use

D. It does not perform bounds checking, which can lead to buffer overflow if input exceeds the array size

Ans: D gets() does not check the size of the destination array, so input longer than the allocated buffer can overflow into adjacent memory, causing undefined behavior or security vulnerabilities.

Q113 Statement-Based gets() - Historical Usage/Status

Which statement best describes the modern status of gets() in the C standard?

A. gets() is the recommended modern function for all string input

B. gets() was never part of any C standard

C. gets() has been officially removed from the C11 standard due to security concerns over buffer overflow

D. gets() automatically prevents buffer overflow in all compilers

Ans: C Due to its inherent buffer overflow risk (no bounds checking), gets() was deprecated in C99 and officially removed from the C11 standard, with fgets() recommended instead.

Q114 Comparison gets() vs scanf() with %s

What is the key advantage of gets() over scanf("%s", ...) when reading text?

A. Both behave identically for all input

B. scanf() with %s can read spaces but gets() cannot

C. gets() requires a loop, scanf() does not

D. gets() can read a full line including spaces, whereas scanf() with %s stops at the first whitespace

Ans: D gets() reads an entire line of input including spaces until a newline, while scanf() with %s only reads a single token and stops at the first whitespace character.

Q115 Fill in the Blank gets() - Header File

Fill in the blank: gets() is declared in the _____ header file.

A. stdlib.h

B. conio.h

C. stdio.h

D. ctype.h

Ans: C gets() belongs to the standard C I/O library declared in stdio.h.

Q116 Code Snippet gets() - Output Tracing

If the user types 'Hello World' and presses Enter, what is stored in str after the following code?

```
char str[50];
gets(str);
```

A. Nothing is stored

B. Only 'Hello' is stored

C. Only 'World' is stored

D. 'Hello World' is stored as a complete string

Ans: D Since gets() reads everything up to the newline including spaces, the entire input 'Hello World' is stored in the string str.

Q117 Error Identification gets() - Common Mistake

What is a likely problem if the user enters more characters than the declared size of the character array passed to gets()?

A. gets() automatically truncates the extra characters safely

B. scanf() takes over automatically to prevent overflow

C. It may cause a buffer overflow since gets() does not check array bounds

D. The program automatically resizes the array

Ans: C Since gets() has no built-in bounds checking, entering more characters than the array can hold may overwrite adjacent memory, resulting in a buffer overflow.

Q118 Assertion-Reason gets() - Safety

Assertion (A): gets() is considered unsafe for production code.
Reason (R): gets() does not limit the number of characters read, risking buffer overflow.
Choose the correct option:

A. Both A and R are true, and R correctly explains A

B. Both A and R are false

C. A is true, but R is false

D. A is false, but R is true

Ans: A gets() is unsafe precisely because it has no mechanism to limit input length to the destination buffer's size, making R a direct and correct explanation for A.

puts() — 10 Questions

Q119 Definition puts() - Purpose

What is the primary purpose of the puts() function in C?

A. To display a string on the screen followed automatically by a newline

B. To read a single character

C. To read a string from input

D. To compare two strings for equality

Ans: A puts() is a standard output function that writes a given string to the console and automatically appends a newline character at the end.

Q120 Syntax puts() - Syntax

Which of the following shows the correct syntax for using puts()?

A. str = puts();

B. puts("%s", str);

C. puts(str);

D. puts[str];

Ans: C puts() takes a single string argument and is called simply as puts(str); without requiring any format specifier.

Q121 Concept puts() - Header File

Which header file declares the puts() function?

A. stdlib.h

B. string.h

C. stdio.h

D. conio.h

Ans: C puts() is declared in stdio.h, alongside the other standard input/output functions.

Q122 Concept puts() - Newline Behaviour

What happens automatically when puts() outputs a string?

A. A newline character is automatically appended after the string

B. The string is automatically converted to uppercase

C. Nothing extra happens; the behavior is identical to printf()

D. A tab character is automatically inserted before the string

Ans: A Unlike printf(), puts() automatically adds a newline character after printing the string, moving the cursor to the next line.

Q123 Concept puts() - Return Value

What does puts() return on successful execution?

A. The length of the string printed

B. Always 0

C. Nothing, since it is a void function

D. A non-negative value (often the newline character) on success, or EOF on error

Ans: D puts() returns a non-negative integer (implementation may return the newline character or zero) on success, and EOF if a write error occurs.

Q124 Comparison printf() vs puts()

Which scenario is more efficient/appropriate when only a plain string (no formatting) needs to be displayed?

A. scanf(), because it can also display output

B. puts(), because it is simpler and specifically designed for printing plain strings with an automatic newline

C. Neither is appropriate; gets() should be used instead

D. printf(), because it always runs faster than puts()

Ans: B When no formatting or variable substitution is required, puts() is the more appropriate and simpler choice since it is specifically designed for plain string output with an automatic newline.

Q125 Fill in the Blank puts() - Header File

Fill in the blank: puts() is declared in the _____ header file.

A. stdio.h

B. stdlib.h

C. ctype.h

D. conio.h

Ans: A puts() is part of the standard C I/O library and is declared in stdio.h.

Q126 Code Snippet puts() - Output Tracing

What will the following code print?
puts("Hello");
puts("World");

A. Hello and World on two separate lines

B. Hello World with a space

C. An error since puts() needs a format specifier

D. HelloWorld on the same line

Ans: A Since puts() automatically adds a newline after each string, 'Hello' and 'World' are printed on two separate lines.

Q127 Statement-Based puts() - Behaviour

Which of the following statements about puts() is TRUE?

A. puts() accepts only a single string argument and cannot use format specifiers

B. puts() can accept multiple strings in a single call separated by commas

C. puts() requires the & operator before its argument

D. puts() is used only to read input

Ans: A puts() is designed to accept exactly one string argument; it does not support multiple arguments or format specifiers like printf() does.

Q128 **Assertion-Reason** **puts() - Newline**

Assertion (A): puts() is preferable over printf() when only a constant string needs to be printed with a newline.
Reason (R): puts() automatically appends a newline, removing the need to manually include \n.
Choose the correct option:

A. A is false, but R is true

B. Both A and R are false

C. A is true, but R is false

D. Both A and R are true, and R correctly explains A

Ans: D Since puts() automatically adds a newline, it simplifies code when displaying a plain string with a line break, making R a valid justification for A.

Comparison & Mixed Concepts — 13 Questions

Q129 Comparison — Character Input vs String Input

What is the fundamental difference between character input and string input in C's console I/O?

A. Character input is impossible without conio.h

B. String input can only read numbers

C. Character input always requires a format specifier while string input never does

D. Character input (getchar/getch/scanf with %c) reads a single character at a time, while string input (gets/scanf with %s) reads a sequence of characters as one unit

Ans: D Character input functions process one character per call, while string input functions like gets() and scanf() with %s read multiple characters together as a complete word or line.

Q130 Comparison — Character Output vs String Output

How does character output differ from string output in C?

A. Character output (putchar) displays one character per call, while string output (puts/printf with %s) displays a full string in a single call

B. String output can never include a newline

C. Character output requires stdio.h while string output requires conio.h

D. Both work in exactly the same way with no difference

Ans: A putchar() outputs a single character per function call, whereas puts() and printf() with %s output an entire string of characters in just one call.

Q131 Comparison — Formatted vs Unformatted I/O

Which pair correctly represents formatted and unformatted I/O functions respectively?

A. printf()/scanf() are formatted; getchar()/putchar()/gets()/puts() are unformatted

B. getchar()/putchar() are formatted; printf()/scanf() are unformatted

C. All six functions are unformatted

D. All six functions are formatted

Ans: A printf() and scanf() use format strings with specifiers (formatted I/O), while getchar(), putchar(), gets(), and puts() operate directly on characters or strings without format specifiers (unformatted I/O).

Q132 | **Comparison** | **Console Input vs Console Output**

Which of the following correctly classifies the seven functions discussed in console-based I/O into input and output categories?

A. All seven functions are output functions

B. Input: scanf(), getch(), getchar(), gets(); Output: printf(), putchar(), puts()

C. All seven functions are input functions

D. Input: printf(), putchar(); Output: scanf(), getchar()

Ans: B

scanf(), getch(), getchar(), and gets() are used to read data (input functions), while printf(), putchar(), and puts() are used to display data (output functions).

Q133 | **Match the Following** | **Function Classification**

Match each function with its correct category:
1. printf() 2. scanf() 3. getchar() 4. puts()
A. Formatted Output B. Formatted Input C. Unformatted Character Input D. Unformatted String Output

A. 1-D, 2-C, 3-B, 4-A

B. 1-B, 2-A, 3-D, 4-C

C. 1-C, 2-D, 3-A, 4-B

D. 1-A, 2-B, 3-C, 4-D

Ans: D

printf() is formatted output, scanf() is formatted input, getchar() is unformatted character input, and puts() is unformatted string output — matching each function to its correct I/O category.

Q134 | **Match the Following** | **Header Files**

Match each function with the header file commonly required to use it:
1. printf() 2. getch() 3. scanf() 4. puts()
A. conio.h B. stdio.h (output) C. stdio.h (input) D. stdio.h (output, string)

A. 1-B, 2-A, 3-C, 4-D

B. 1-A, 2-B, 3-C, 4-D

C. 1-D, 2-C, 3-B, 4-A

D. 1-C, 2-D, 3-A, 4-B

Ans: A

printf() and puts() are output functions in stdio.h, scanf() is an input function in stdio.h, while getch() is conventionally found in the compiler-specific conio.h header.

Q135 | Difficult/Tricky | Console I/O - Conceptual

Why is it generally NOT recommended to mix scanf("%d", &n) immediately followed by gets(str) in the same program without precaution?

A. Because gets() always overwrites variables read by scanf()

B. Because scanf() deletes the input buffer completely

C. Because the leftover newline character in the input buffer after scanf() can be immediately consumed by gets(), resulting in str appearing empty

D. Because gets() cannot be used after scanf() under any circumstance

Ans: C

After scanf("%d", &n) reads a number, the newline character (from pressing Enter) remains in the input buffer; gets() then reads this leftover newline first, often resulting in an empty string being stored.

Q136 | Previous-Year Pattern | Console I/O - General Concept

Which of the following best summarizes the role of console-based I/O functions like printf(), scanf(), getchar(), getch(), putchar(), gets(), and puts()?

A. They allow a C program to interact with the user through the standard input (keyboard) and standard output (screen) devices

B. They are used only for file handling operations

C. They are exclusively used for mathematical calculations

D. They are used only in graphical user interface programming

Ans: A

These seven functions collectively enable a C program to take input from the keyboard and display output on the screen, forming the foundation of console-based interaction.

Q137 | Statement-Based | Standard vs Non-Standard Functions

Which of the following functions among the seven discussed is generally considered NON-standard (compiler-specific) rather than part of ANSI C?

A. getch()

B. puts()

C. printf()

D. scanf()

Ans: A

getch() is not part of the ANSI C standard library; it is a compiler-specific function commonly available via conio.h in environments like Turbo C, unlike the other six which are standard.

Q138 Scenario-Based Console I/O - Practical Application

A program needs to: (1) ask the user's name as a full line of text, and (2) display it back. Which pair of functions is most appropriate?

A. gets() and puts()

B. scanf("%d") and printf()

C. scanf("%c") and putchar()

D. getch() and getchar()

Ans: A Since the full name (potentially with spaces) needs to be read as a line and then displayed, gets() (to read the line) and puts() (to display it with an automatic newline) are most appropriate.

Q139 Difficult/Tricky Console I/O - Buffer Concept

What general programming concept explains most input-buffer-related issues when mixing different console I/O functions (e.g., scanf() then getchar())?

A. C programs do not use any buffer for input; input is always processed character-by-character with no leftover

B. Each input function automatically clears the buffer before and after execution

C. Characters not yet consumed (like a leftover newline) remain in the input stream buffer and are picked up by the next input function call

D. The buffer is reset every time printf() is called

Ans: C Standard input is buffered, meaning unconsumed characters (such as a trailing newline left after scanf()) stay in the buffer and get read by the very next input function, often causing unexpected behavior.

Q140 Error Identification Console I/O - Debugging

A student writes: int n; scanf("%d" &n); — what is the error in this line?

A. The & symbol should not be used at all

B. A comma is missing between the format string "%d" and &n

C. There is no error; the line is correct

D. %d should be replaced with %c

Ans: B In a function call, all arguments must be separated by commas; the missing comma between "%d" and &n results in a syntax error.

Q141	Concept	Console I/O - Standard Streams

Which standard streams are involved when using printf() and scanf() respectively?

A. Both functions use only standard error (stderr)

B. Neither function is associated with any stream

C. printf() uses standard output (stdout); scanf() uses standard input (stdin)

D. printf() uses standard input; scanf() uses standard output

Ans: C

printf() writes data to the standard output stream (stdout), while scanf() reads data from the standard input stream (stdin), which are the console screen and keyboard respectively by default.